

Data Programming in Python | BAIS:6040

Homework 4. Files and Pandas

File Handling Questions (15 points)

You have a dataframe `df` with the features of diamonds.

```
In [1]: # Do not make any change to this cell. Just run this cell.
import os
from seaborn import load_dataset

df = load_dataset("diamonds")
df
```

```
Out[1]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 10 columns

1. (5 pt) Create a CSV file named `hw_diamonds.csv` in the `outcome` directory. The file contains 7 column names `carat`, `cut`, `color`, `clarity`, `depth`, `table`, and `price` in the first row and the values from the corresponding columns of `df` in the following rows. The delimiter is a slash (/) for columns and a new line (\n) for rows. For example, the first 3 rows in the file must look as below. Do not use `pandas.to_csv`. Hint - loop through each row of the dataframe above using `df.iterrows()`. Use format printing `.format` or f-strings.

<https://www.geeksforgeeks.org/python/python-output-formatting/>

carat/cut/color/clarity/depth/table/price 0.23/Ideal/E/SI2/61.5/55.0/326 0.21/Premium/E/SI1/59.8/61.0/326

```
In [4]: # Your answer here
columns = ['carat', 'cut', 'color', 'clarity', 'depth', 'table', 'price']
with open("outcome/hw_diamonds.csv", 'w') as f:
    # Write header
    f.write('/'.join(columns) + '\n')

    # Loop through each row in the DataFrame
    for index, row in df.iterrows():
        # Create a slash-separated string of values from the specified columns
        line = '/'.join(str(row[col]) for col in columns)
        f.write(f"{line}\n")
```

```
In [5]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
os.path.isfile("outcome/hw_diamonds.csv") # Expected output: True
```

Out[5]: True

2. (5 pt) Read the file `hw_diamonds.csv` created in Question 1 and print the first 10 rows in the file including the header row, separated by a comma (,) between columns. For example, the output must be printed as below. Do not use Pandas to read in the file.

carat,cut,color/clarity/depth/table/price 0.23,Ideal,E,Si2,61.5,55.0,326 0.21,Premium,E,Si1,59.8,61.0,326
 0.23,Good,E,VS1,56.9,65.0,327 0.29,Premium,I,VS2,62.4,58.0,334 0.31,Good,J,Si2,63.3,58.0,335 0.24,Very
 Good,J,VVS2,62.8,57.0,336 0.24,Very Good,I,VVS1,62.3,57.0,336 0.26,Very Good,H,Si1,61.9,55.0,337
 0.22,Fair,E,VS2,65.1,61.0,337

```
In [ ]: # Your answer here
with open("outcome/hw_diamonds.csv", 'r') as fh:
    # Read all lines
    lines = fh.readlines()

    # Print the first 10 rows
    for line in lines[:10]:
        print(line.strip().replace('/', ','))
```

```
carat, cut, color, clarity, depth, table, price
0.23, Ideal, E, SI2, 61.5, 55.0, 326
0.21, Premium, E, SI1, 59.8, 61.0, 326
0.23, Good, E, VS1, 56.9, 65.0, 327
0.29, Premium, I, VS2, 62.4, 58.0, 334
0.31, Good, J, SI2, 63.3, 58.0, 335
0.24, Very Good, J, VVS2, 62.8, 57.0, 336
0.24, Very Good, I, VVS1, 62.3, 57.0, 336
0.26, Very Good, H, SI1, 61.9, 55.0, 337
0.22, Fair, E, VS2, 65.1, 61.0, 337
```

3. (5 pt) Create a list `diamonds`, in which each element is a dictionary with 3 keys being `carat`, `cut`, and `color` and the values being their corresponding values in `df`. For example, the first element in `diamonds` must look as below. Then, serialize the first 1,000 items in `diamonds` as a JSON formatted stream to a file named `hw_diamonds.json` saved in the `outcome` directory.

```
{'carat': 0.23, 'cut': 'Ideal', 'color': 'E'}
```

```
In [7]: # Your answer here
import os
import json

diamonds = []

for idx, row in df.iterrows():
    diamond = {
        'carat': row['carat'],
        'cut': row['cut'],
        'color': row['color']
    }
    diamonds.append(diamond)

with open("outcome/hw_diamonds.json", "w") as fh:
    json.dump(diamonds[:1000], fh, indent=2)
```

```
In [8]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
diamonds[0]
```

```
Out[8]: {'carat': 0.23, 'cut': 'Ideal', 'color': 'E'}
```

```
In [9]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
len(diamonds)                                     # Expected output: 53940
```

```
Out[9]: 53940
```

```
In [10]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
os.path.isfile("outcome/hw_diamonds.json")        # Expected output: True
```

```
Out[10]: True
```

Pandas Questions (15 points)

You have a dataframe `df` in which each row refers to some features of a diamond such as carat, cut, color, clarity, depth, table, price, x, y, and z.

```
In [ ]: # Do not make any change to this cell. Just run this cell.
import numpy as np
from seaborn import load_dataset

df = load_dataset("diamonds")
np.random.seed(0)
df.iloc[np.random.randint(0, len(df), 10), :] = None
```

1. Get the list of column names of `df` and save it in the variable `ans1`. The type of the list does not have to be the primitive Python list.

```
In [26]: # Your answer here
ans1 = df.columns
```

```
In [28]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans1
```

```
Out[28]: Index(['carat', 'cut', 'color', 'clarity', 'depth', 'table', 'price', 'x', 'y',
               'z'],
              dtype='object')
```

2. Select the first 10 rows from `df` and save the resulting dataframe in the variable `ans2`.

```
In [27]: # Your answer here
ans2 = df.head(10)
```

```
In [29]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans2
```

```
Out[29]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39

3. Select all values in the `cut` column and save the resulting series in the variable `ans3`.

```
In [30]: # Your answer here
ans3 = df.cut
```

```
In [31]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans3
```

```
Out[31]: 0          Ideal
1        Premium
2          Good
3        Premium
4          Good
...
53935       Ideal
53936        Good
53937    Very Good
53938        Premium
53939       Ideal
Name: cut, Length: 53940, dtype: category
Categories (5, object): ['Ideal', 'Premium', 'Very Good', 'Good', 'Fair']
```

4. Select all rows in `df` that meet all the following three conditions: the cut is *Premium*, the color is G, and the price is greater than 10,000. Save the resulting dataframe in the variable `ans4`.

```
In [32]: # Your answer here
ans4 = df[(df.cut=="Premium") & (df.color=='G') & (df.price>10000)]
```

```
In [33]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans4
```

Out[33]:

	carat	cut	color	clarity	depth	table	price	x	y	z
21939	1.35	Premium	G	VS1	62.1	59.0	10012	7.06	7.02	4.37
21951	1.51	Premium	G	SI1	58.7	60.0	10020	7.55	7.48	4.41
21953	1.51	Premium	G	SI1	60.6	58.0	10020	7.47	7.39	4.50
21954	1.51	Premium	G	SI1	59.7	58.0	10020	7.50	7.45	4.46
21955	1.51	Premium	G	SI1	62.0	58.0	10020	7.31	7.26	4.52
...	...	...	...	...	...	...	...	...	...	...
27670	2.03	Premium	G	VS2	59.6	60.0	18680	8.27	8.21	4.91
27680	2.01	Premium	G	SI2	61.2	57.2	18705	8.08	8.14	4.97
27687	1.70	Premium	G	VVS2	59.8	59.0	18718	7.70	7.75	4.62
27689	1.72	Premium	G	VVS2	61.8	60.0	18730	7.65	7.68	4.74
27725	2.01	Premium	G	SI1	60.3	59.0	18741	8.23	8.15	4.94

365 rows × 10 columns

5. Select all rows in `df` that have any null values. Save the resulting dataframe in the variable `ans5`.

```
In [37]: # Your answer here
ans5 = df[df.isnull().any(axis=1)]
```

```
In [39]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans5
```

```
Out[39]:   carat  cut  color  clarity  depth  table  price  x  y  z
```

6. Fill all the null values in `df` such that the last non-null observations are propagated forward. Make sure to update `df`.

```
In [40]: # Your answer here
df.fillna(method="ffill")
```

C:\Users\mredmond\AppData\Local\Temp\ipykernel_4768\742381370.py:2: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
df.fillna(method="ffill")

Out[40]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 10 columns

In []: *# Check your answer here. (Do not make any change to this cell. Just run this cell.*
df.info()

7. Get the mean carat, mean depth, mean table, and mean price of all diamonds and save them in the variables `ans7_1`, `ans7_2`, `ans7_3`, and `ans7_4`, respectively.

In [41]: *# Your answer here*
`ans7_1, ans7_2, ans7_3, ans7_4 = df.carat.mean(), df.depth.mean(), df.table.mean(),`

In [42]: *# Check your answer here. (Do not make any change to this cell. Just run this cell.*
`ans7_1, ans7_2, ans7_3, ans7_4`

Out[42]: (0.7979397478680014, 61.749404894327036, 57.45718390804598, 3932.799721913237)

8. Get the mean carat, mean depth, mean table, and mean price of premium-cut diamonds and save them in the variables `ans8_1`, `ans8_2`, `ans8_3`, and `ans8_4`, respectively.

In [43]: *# Your answer here*
`df_prem = df[df.cut=='Premium']`

`ans8_1 = df_prem.carat.mean()`
`ans8_2 = df_prem.depth.mean()`
`ans8_3 = df_prem.table.mean()`
`ans8_4 = df_prem.price.mean()`

In [44]: *# Check your answer here. (Do not make any change to this cell. Just run this cell.*
`ans8_1, ans8_2, ans8_3, ans8_4`

Out[44]: (0.8919548981219638, 61.26467261257343, 58.74609527953013, 4584.2577042999055)

9. Group `df` by `color` to get the mean price for each color. Save the resulting series in the variable `ans9`.

```
In [45]: # Your answer here
ans9 = df.groupby("color").price.mean()
```

C:\Users\mredmond\AppData\Local\Temp\ipykernel_4768\3479311390.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
ans9 = df.groupby("color").price.mean()
```

```
In [ ]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans9
```

10. Sort the `df` by the columns `carat` and `price` both in ascending order, but do not change `df`. Save the resulting dataframe in the variable `ans10`.

```
In [46]: # Your answer here
ans10 = df.sort_values(by=["carat", "price"], ascending = True)
```

```
In [47]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans10
```

```
Out[47]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
14	0.20	Premium	E	SI2	60.2	62.0	345	3.79	3.75	2.27
31591	0.20	Premium	E	VS2	59.8	62.0	367	3.79	3.77	2.26
31592	0.20	Premium	E	VS2	59.0	60.0	367	3.81	3.78	2.24
31593	0.20	Premium	E	VS2	61.1	59.0	367	3.81	3.78	2.32
31594	0.20	Premium	E	VS2	59.7	62.0	367	3.84	3.80	2.28
...	...	...	...	...	...	...	...	...	...	...
25998	4.01	Premium	I	I1	61.0	61.0	15223	10.14	10.10	6.17
25999	4.01	Premium	J	I1	62.5	62.0	15223	10.02	9.94	6.24
27130	4.13	Fair	H	I1	64.8	61.0	17329	10.00	9.85	6.43
27630	4.50	Fair	J	I1	65.8	58.0	18531	10.23	10.16	6.72
27415	5.01	Fair	J	I1	65.5	59.0	18018	10.74	10.54	6.98

53940 rows × 10 columns

```
In [ ]:
```